

TECH CHALLENGE — PÓS-GRADUAÇÃO EM ARQUITETURA DE SOFTWARE

⚡ FASE 2 — INFRAESTRUTURA & ESCALABILIDADE



Mecânica API

Evolução para Infraestrutura Escalável e Cloud-Native

Entrega Fase 2 — Infraestrutura, CI/CD e Escalabilidade

Grupo 14SOAT | Turma SOAT — FIAP

Período: 2025 / 2026 · Entrega: Março 2026

Docker Multi-Stage

Kubernetes EKS 1.33

Terraform IaC

GitHub Actions CI/CD

HPA: min 1 / max 10 pods

HMAC-SHA256 Email

API Key M2M

4 Novos Endpoints

EQUIPE — GRUPO 14SOAT

CF

Cleber Fernandes

Software Engineer

CL

Christian Felipe F. de Lima

Software Engineer

GS

Gabriel J. A. Schuina

Software Engineer

Links e Artefatos de Entrega

Repositório · Swagger · Insomnia · README · Vídeo

REPOSITÓRIO GITHUB

<https://github.com/clefern/fiap-tc-mecanica-java>

Código-fonte completo com Fase 1 + Fase 2 · Branch main (produção) · Histórico de PRs e pipelines

SWAGGER UI — DOCUMENTAÇÃO DAS APIS

<http://localhost:8080/swagger-ui.html>

OpenAPI 3.0 · 50+ endpoints documentados · Inclui todos os endpoints novos da Fase 2

COLLECTION INSOMNIA

[docs/api/Insomnia_export.yaml](#)

Fluxo E2E completo + endpoints por categoria · Incluindo novos endpoints da Fase 2

README.MD — GUIA COMPLETO FASE 2

[README.md](#) (raiz do repositório)

Seção Fase 2: diagrama AWS, instruções de deploy K8s, comandos Terraform, variáveis de ambiente

VÍDEO DEMONSTRATIVO — YOUTUBE

<https://youtu.be/sCPwD91CEW0>

Deploy na AWS, execução do CI/CD, consumo das APIs (Swagger), escalabilidade via HPA

DESENHO C4 MODEL

[docs/entrega/fase-2/c4.txt](#)

Este desenho representa até o nível C2

Pode ser utilizado pelo site structurizr

COMO EXECUTAR — DESENVOLVIMENTO LOCAL

- ▶ `make dev` — sobe Docker Compose completo
- ▶ `make test` — roda 1052+ testes
- ▶ `make integration-test` — testes com DB real
- ▶ `make coverage` — relatório JaCoCo

COMO FAZER DEPLOY — KUBERNETES

- ▶ `cd infra/conf && terraform apply`
- ▶ Configurar `kubectl` com output do Terraform
- ▶ `kubectl apply -f k8s/base/`
- ▶ Verificar: `kubectl get pods,hpa`

Escopo da Fase 2: Evolução da aplicação backend com refatoração arquitetural, novos endpoints de negócio, containerização avançada, orquestração Kubernetes na AWS, infraestrutura como código via Terraform e pipeline CI/CD completo — entrega como sistema cloud-native de produção.

Requisitos vs Entregáveis

Mapeamento completo com evidências — 20 de 22 requisitos implementados no código

EVOLUÇÃO DA APLICAÇÃO

Requisito	Status	Evidência
Refatoração: Clean Code + Arquitetura Hexagonal	✓ OK	refactor/os-dependencies — OrdemServiceServiceImpl decomposto; 15 interfaces *Api
Testes automatizados para fluxos críticos	✓ OK	1052+ testes; JaCoCo ≥ 90% global / ≥ 80% domínio
Abertura de OS (cliente + veículo + serviços + peças)	✓ OK	POST /api/ordens-servico/abertura-completa
Consulta de status da OS	✓ OK	GET /api/ordens-servico/{id}/status
Aprovação de orçamento via notificação externa	✓ OK	POST /api/integracoes/orcamentos/aprovacao — API key M2M
Listagem de OS com ordenação por status + exclusão FINALIZADA/ENTREGUE	✓ OK	GET /api/ordens-servico/fila-operacional
Atualização de status via email	✓ OK	Links HMAC-SHA256 no email de orçamento — ActionTokenService

INFRAESTRUTURA

Requisito	Status	Evidência
Dockerfile atualizado	✓ OK	/Dockerfile (Alpine) + /Dockerfile.deploy (distroless)
docker-compose para desenvolvimento local	✓ OK	/docker-compose.yml — 5 serviços
Manifestos K8s: Deployments, Services, ConfigMaps, Secrets	✓ OK	/k8s/base/ + overlays lab/prod
HPA escalando por CPU/memória	✓ OK	/k8s/base/hpa.yaml — CPU 70% / Mem 80%, min 1, max 10
Terraform — Cluster Kubernetes (EKS)	✓ OK	/infra/conf/ — VPC, EKS 1.33, node group t3.large
Terraform — Banco de Dados	⚠ PVC	PostgreSQL StatefulSet + PVC 10Gi EBS (não RDS por custo de laboratório)

CI/CD E DOCUMENTAÇÃO

Requisito	Status	Evidência
CI/CD — Build, testes, Docker, deploy K8s	✓ OK	<code>.github/workflows/</code> — 5 workflows (ci, test, build, cd, infra)
README.md com arquitetura, instruções de deploy e Terraform	✓ OK	README.md — diagrama ASCII AWS, make commands, guias completos
Link para collection das APIs (Swagger/Postman)	✓ OK	<code>docs/api/Insomnia_export.yaml</code> + Swagger em <code>/swagger-ui.html</code>
Vídeo demonstrativo (YouTube/Vimeo, ≤ 15 min)	✓ OK	youtu.be/sCPwD91CEW0
PDF de entrega com link GitHub + arquitetura + link vídeo	✓ OK	Este documento — <code>docs/entrega/fase-2/entrega-fase2-grupo14soat.pdf</code>

Nota sobre banco de dados: PostgreSQL roda como StatefulSet no Kubernetes com PVC de 10Gi no EBS. A arquitetura está preparada para migração a RDS alterando apenas a string de conexão e os secrets — sem mudança de código.

Refatoração e Qualidade de Código

Clean Code · Arquitetura Hexagonal reforçada · 1052+ testes passando

REFATORAÇÃO APLICADA (CLEAN CODE)

- ▶ **OrdemServicoServiceImpl decomposto** — classe monolítica dividida em serviços coesos com responsabilidade única
- ▶ **15 interfaces** extraídas para contratos OpenAPI separados dos controllers `*Api`
- ▶ **Nomes claros** — renomeação de métodos e variáveis para linguagem ubíqua do domínio
- ▶ **Coesão por use case** — cada service impl cuida de um bounded context
- ▶ **JWT secret** removido do application.yml base — obrigatório via env var em produção
- ▶ **Token Sonar** hardcoded removido do código (ADR-SEC-001)

ARQUITETURA HEXAGONAL REFORÇADA

- ▶ Domain layer: **zero anotações Spring/JPA** — POJO puro verificado
- ▶ Ports em `domain/repos itory/`, `infra/adap ter/` adapters em
- ▶ Dois mappers separados: infra (domain↔entity) e presentation (domain↔DTO)
- ▶ Integração M2M isolada em `IntegracaoOrcamentoCon troller`
- ▶ API Key filter separado do JWT filter — responsabilidades distintas
- ▶ 282 classes Java mantendo separação estrita de camadas

TESTES AUTOMATIZADOS — FLUXOS CRÍTICOS COBERTOS

Unitários

- ▶ Máquina de estados da OS
- ▶ Validações de orçamento
- ▶ Regras de estoque
- ▶ Value Objects (CPF, CNPJ, Email)
- ▶ Domain events

Integração (TestContainers)

- ▶ Repositórios JPA com PostgreSQL real
- ▶ Controllers completos (MockMvc)
- ▶ Fluxo abertura completa de OS
- ▶ Aprovação via token HMAC
- ▶ Fila operacional filtrada

Qualidade & Análise

- ▶ JaCoCo ≥ 90% global / ≥ 80% domínio
- ▶ Checkstyle (Google Java Style)
- ▶ PMD — complexidade ciclomática
- ▶ SpotBugs — null/resource leaks
- ▶ SonarQube contínuo

MELHORIAS DE PERFORMANCE — FASE 2

@BatchSize(20)

Em OrdemServicoEntity.itens — evita N+1 em listagens paginadas

V16 — 3 Índices

idx_status, idx_status+data, idx_orcamentos — aceleração de queries críticas

Spring Cache

@Cacheable em endpoints de consulta de alta frequência

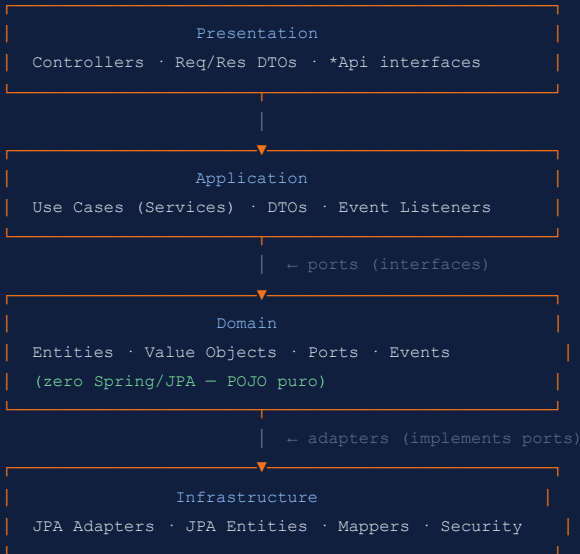
Novos Endpoints Implementados

4 endpoints + fluxo de aprovação por email com token seguro

ENDPOINTS — VISÃO GERAL

Método	Endpoint	Auth	Requisito Atendido
POST	/api/ordens-servico/abertura-completa	JWT Atendente	Abertura de OS com cliente, veículo, serviços e peças em um request
GET	/api/ordens-servico/fila-operacional	JWT Mecânico	Listagem ordenada por prioridade (Execução > Ag.Apr. > Diagnóstico > Recebida)
GET	/api/ordens-servico/{id}/status	JWT — todos roles	Consulta de status resumido — DTO enxuto para polling de frontend/cliente
POST	/api/integracoes/orcamentos/aprovacao	API Key M2M	Aprovação/recusa de orçamento via integração externa (X-API-Key header)
GET	/api/integracoes/orcamentos/aprovacao?token=...	HMAC Token	Aprovação via link do email — sem necessidade de login JWT

ARQUITETURA DA APLICAÇÃO — HEXAGONAL + DDD



TESTES AUTOMATIZADOS — FLUXOS CRÍTICOS COBERTOS

Unitários

- ▶ Máquina de estados da OS
- ▶ Validações de orçamento
- ▶ Regras de estoque
- ▶ Value Objects (CPF, CNPJ, Email)
- ▶ Domain events

Integração (TestContainers)

- ▶ Repositórios JPA com PostgreSQL real
- ▶ Controllers completos (MockMvc)
- ▶ Fluxo abertura completa de OS
- ▶ Aprovação via token HMAC
- ▶ Fila operacional filtrada

Qualidade & Análise

- ▶ JaCoCo $\geq 90\%$ global / $\geq 80\%$ domínio
- ▶ Checkstyle (Google Java Style)
- ▶ PMD — complexidade ciclomática
- ▶ SpotBugs — null/resource leaks
- ▶ SonarQube contínuo

Docker + Kubernetes

Imagens otimizadas · Manifests K8s completos · HPA automático

DOCKER — BUILD MULTI-STAGE

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS builder
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime (Alpine ~120MB)
FROM eclipse-temurin:21-jre-alpine
COPY --from=builder /app/target/*.jar app.jar
USER nonroot
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- ▶ **Dockerfile** — JRE Alpine (~120MB, cache Maven)
- ▶ **Dockerfile.deploy** — distroless (zero shell, máxima segurança)
- ▶ Usuário não-root em todas as imagens
- ▶ Trivy scan integrado no pipeline

DOCKER COMPOSE — 5 SERVIÇOS

Serviço	Porta	Função
mecanica-ap p	8080	API + Swagger
mecanica-po stgres	5433	PostgreSQL 16
mecanica-ad miner	8081	DB Browser
mecanica-ma ilhog	8025	Email catcher
sonarqube	9000	Code quality

make dev — sobe todos os serviços com rebuild automático

KUBERNETES — MANIFESTS COMPLETOS (/K8S)

k8s/base/

Manifest	Conteúdo
namespace.yaml	Namespace isolado
configmap.yaml	Variáveis não-sensíveis
secret.yaml	DB, JWT, API Key (base64)
deployment.yaml	App + liveness/readiness probes
service.yaml	ClusterIP + LoadBalancer
hpa.yaml	HPA CPU 70% / Mem 80%
pvc.yaml	Persistent Volume 10Gi (EBS)

HPA — Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
spec:
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - cpu: 70%
    - memory: 80%
```

Overlays separados em k8s/overlays/lab/ e k8s/overlays/prod/

AWS + Terraform

VPC · EKS 1.33 · ECR · ALB · NGINX · Cert-Manager — tudo como código

DOCKER — BUILD MULTI-STAGE

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS builder
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime (Alpine ~120MB)
FROM eclipse-temurin:21-jre-alpine
COPY --from=builder /app/target/*.jar app.jar
USER nonroot
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- ▶ **Dockerfile** — JRE Alpine (~120MB, cache Maven)
- ▶ **Dockerfile.deploy** — distroless (zero shell, máxima segurança)
- ▶ Usuário não-root em todas as imagens
- ▶ Trivy scan integrado no pipeline

DOCKER COMPOSE — 5 SERVIÇOS

Serviço	Porta	Função
mecanica-ap p	8080	API + Swagger
mecanica-po stgres	5433	PostgreSQL 16
mecanica-ad miner	8081	DB Browser
mecanica-ma ilhog	8025	Email catcher
sonarqube	9000	Code quality

make dev — sobe todos os serviços com rebuild automático

KUBERNETES — MANIFESTS COMPLETOS (/K8S)

k8s/base/

Manifest	Conteúdo
namespace.yaml	Namespace isolado
configmap.yaml	Variáveis não-sensíveis
secret.yaml	DB, JWT, API Key (base64)
deployment.yaml	App + liveness/readiness probes
service.yaml	ClusterIP + LoadBalancer
hpa.yaml	HPA CPU 70% / Mem 80%
pvc.yaml	Persistent Volume 10Gi (EBS)

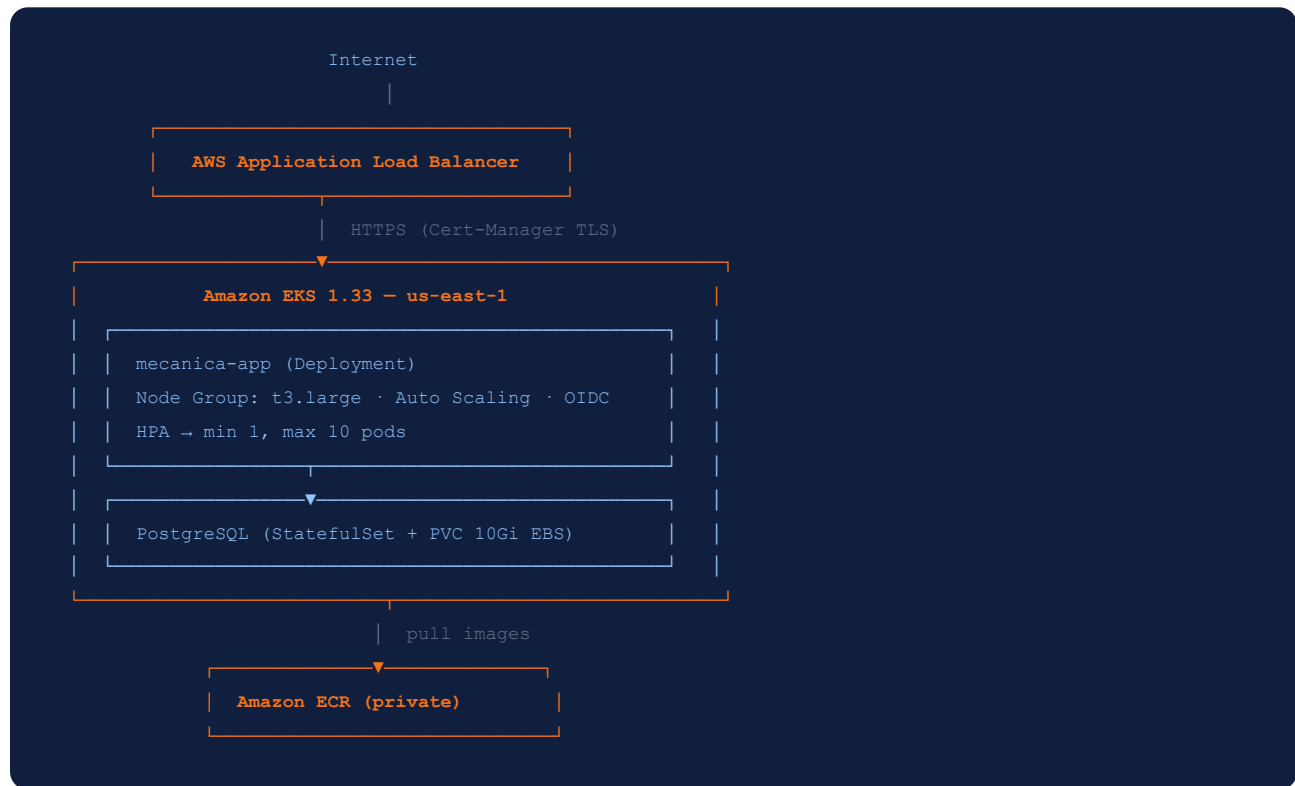
HPA — Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
spec:
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - cpu: 70%
    - memory: 80%
```

Overlays separados em k8s/overlays/lab/ e k8s/overlays/prod/

AWS + Terraform

VPC · EKS 1.33 · ECR · ALB · NGINX · Cert-Manager — tudo como código



VPC & NETWORK (TF-001)

- ▶ VPC customizada com CIDR definido
- ▶ Subnets públicas e privadas
- ▶ Internet Gateway + NAT Gateway
- ▶ Route tables configuradas
- ▶ Security Groups por camada

SERVIÇOS AWS (TF-002)

- ▶ Amazon ECR — repositório privado
- ▶ Amazon S3 — arquivos estáticos e tfstate
- ▶ Amazon DynamoDB — tabela de locks
- ▶ Amazon EKS — cluster v1.33
- ▶ Amazon ASG — t3.large
- ▶ IAM roles

SERVIÇOS K8S (TF-003)

- ▶ ALB Controller via Helm
- ▶ NGINX Ingress Controller
- ▶ Metrics Server (HPA dependency)
- ▶ Cert-Manager (TLS automático)
- ▶ EBS CSI Driver (via OIDC ou Pod Identity)

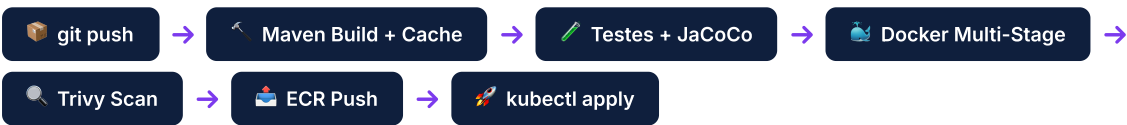
ESTRUTURA TERRAFORM (/INFRA)

- ▶ `environments/` — entry points por ambiente (development, lab, local)
- ▶ `conf/0-providers.tf` — provider AWS
- ▶ `conf/variables.tf` — region, eks_name, env, zones
- ▶ `conf/output.tf` — vpc_id, cluster_name, ecr_repository
- ▶ `conf/2-vpc..6-routes` — VPC, subnets, IGW, NAT, routes
- ▶ `conf/7-eks.tf`, `8-nodes.tf` — cluster + node group
- ▶ `conf/14-ecr.tf` — repositório de imagens
- ▶ `conf/10-13` — Helm: metrics-server, ALB, Nginx, cert-manager
- ▶ `conf/15-ebs-csi`, `16-oidc.tf` — EBS CSI + OIDC
- ▶ **Uso:** `terraform init && terraform apply` no `env`

AUTOMAÇÃO

Pipeline CI/CD — GitHub Actions

5 workflows separados · do push ao deploy em EKS em produção



5 WORKFLOWS GITHUB ACTIONS

Workflow	Trigger	Arquivo	Responsabilidade
CI	Push/PR → develop	ci.yml	Compilação Maven, cache M2, lint (Checkstyle + PMD + SpotBugs)
Test	Push/PR → develop	test.yml	Testes unitários e integração (TestContainers), cobertura JaCoCo
Build	Merge → main	build.yml	Docker multi-stage + Trivy vulnerability scan + push para ECR
CD	Após build	cd.yml	Deploy em EKS via <code>kubect l apply + k8s-depl oy.sh</code> (envsubst)
Infra	Manual dispatch	infra.yml	Terraform plan/apply — provisionamento da infraestrutura AWS

SEGURANÇA NO PIPELINE

- ▶ OIDC para autenticação AWS — sem chaves estáticas de longa duração
- ▶ Secrets via GitHub Actions Secrets (não expostos em logs)
- ▶ Trivy: scan de CVEs na imagem Docker antes do push
- ▶ EBS CSI Driver com OIDC provider do EKS
- ▶ Imagens distroless em produção — zero shell disponível

GITHUB SECRETS CONFIGURADOS

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_SESSION_TOKEN
ECR_REGISTRY / ECR_REPOSITORY
EKS_CLUSTER_NAME
JWT_SECRET / INTEGRATION_API_KEY
DB_URL / DB_USERNAME / DB_PASSWORD
```